

Experiment No. 3

Title:

Design Patterns in Python

Aim:

Write a program to implement a Configurable Payment Processing System Using Strategy Pattern.

Theory:

This program demonstrates a payment processing system where different payment methods (e.g., credit card, PayPal, Bitcoin) can be used interchangeably. The Strategy Pattern is employed to define a family of algorithms (payment methods), encapsulate each one, and make them interchangeable. This pattern allows the payment method to be selected at runtime.

- **Strategy Pattern:** This design pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.
- **Payment Processing System:** A system that handles transactions from various payment methods, ensuring flexibility and scalability.

What are Design Patterns?

Design patterns in Python are communicating objects and classes that are customized to solve a general design problem in a particular context. Software design patterns are general, reusable solutions to common problems that arise during the design and development of software. They represent best practices for solving certain types of problems and provide a way for developers to communicate about effective design solutions.

A design pattern is a generic repeatable solution to a frequently occurring problem in software design that is used in software engineering. It isn't a complete design that can be written in code right away. It is a description or model for problem-solving that may be applied in a variety of contexts.

Types of Software Design Patterns in Python

There are mainly three types of Design Patterns in Python:

1. Creational Design Patterns in Python

Creational design patterns are a subset of design patterns in software development. They deal with the process of object creation, trying to make it more flexible and efficient. It makes the system independent and how its objects are created, composed, and represented.

Types of Creational Design Patterns in Python:

1.1 Factory Method

Factory Method is a creational design pattern, that provide an interface for creating objects in superclass, but subclasses are responsible to create the instance of the class.

1.2 Abstract Factory Method

Abstract Factory Method is a creational design pattern, it provides an interface for creating families of related or dependent objects without specifying their concrete classes.

1.3 Builder Method

Builder Method is a creational design pattern, it provides an interface for constructing an object and then have concrete builder classes that implement this interface to create specific objects in a stepwise manner.

1.4 Prototype Method

Prototype Method is a creational design pattern, it provide to create new objects with the same structure and initial state as an existing object without explicitly specifying their class or construction details.

1.5 Singleton Method

Singleton Method is a creational design pattern, it provide a class has only one instance, and that instance provides a global point of access to it.

2. Structural Design Patterns in Python

Structural design patterns are a subset of design patterns in software development that focus on the composition of classes or objects to form larger, more complex structures. They help in organizing and managing relationships between objects to achieve greater flexibility, reusability, and maintainability in a software system.

Types of Structural Design Patterns in Python:

2.1 Adapter Method

Adapter Method is a structural design pattern, it allows you to make two incompatible interfaces work together by creating a bridge between them.

2.2 Bridge Method

Bridge Method is a structural design pattern, it provide to design separate an object's abstraction from its implementation so that the two can vary independently.

2.3 Composite Method

Composite Method is structural design pattern, it's used to compose objects into tree structures to represent part-whole hierarchies. This pattern treats both individual objects and compositions of objects it allow clients to work with complex structures of objects as if they were individual objects.

2.4 Decorator Method

Decorator Method is structural design pattern, it allows to add behavior to individual objects, either statically or dynamically, without affecting the behavior of other objects from the same class.

2.5 Facade Method

Facade Method is a structural design pattern, it provides a simplified, higher-level interface to a set of interfaces in a subsystem, making it easier for clients to interact with that subsystem.

2.6 Proxy Method

Proxy Method is a structural design pattern, it provide to create a substitute for an object, which can act as an intermediary or control access to the real object.

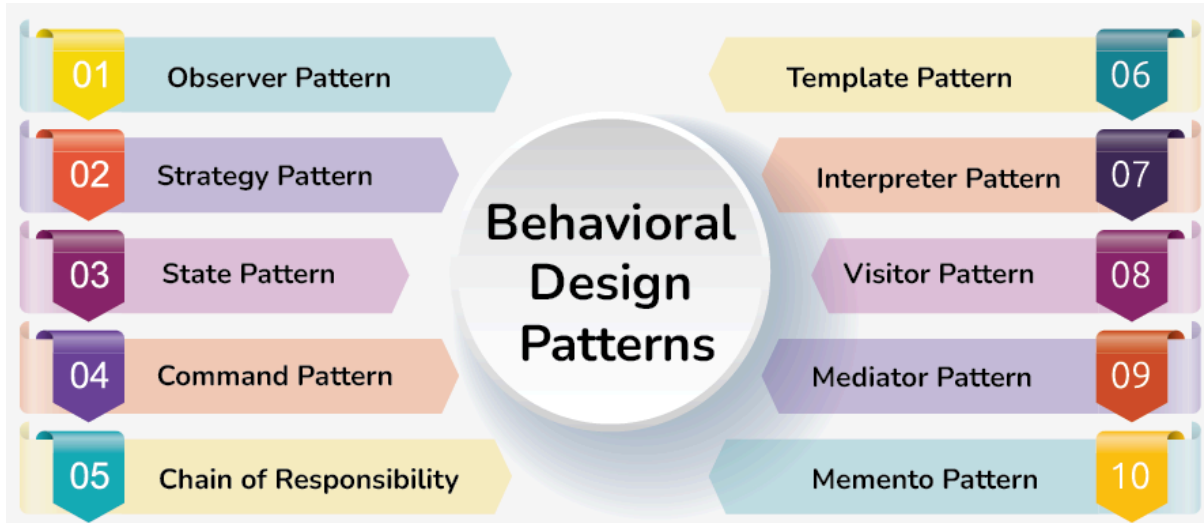
2.7 Flyweight Method

Flyweight Method is a structural design pattern, it is used when we need to create a lot of objects of a class. Since every object consumes memory space that can be crucial for low memory devices, flyweight design pattern can be applied to reduce the load on memory by sharing objects.

3. Behavioral Design Patterns in Python

Behavioral design patterns are a subset of design patterns in software development that deal with the communication and interaction between objects and classes. They focus on how objects and classes collaborate and communicate to accomplish tasks and responsibilities.

Types of Behavioral Design Pattern in Python:



3.1 Command Method

Command Method is a Behavioral Design Pattern, it promotes loose coupling between the sender (client) and the receiver (the object that performs the operation) and provides a way to support undoable operations.

3.2. Observer Method

It defines a one-to-many dependency between objects, so that when one object (the subject) changes its state, all its dependents (observers) are notified and updated automatically.

3.3 Mediator Method

Mediator Method is a Behavioral Design Pattern, it promotes loose coupling between objects by centralizing their communication through a mediator object. Instead of objects directly communicating with each other, they communicate through the mediator, which encapsulates the interaction and coordination logic.

3.4 Memento Method

Momento Method is a Behavioral Design Pattern, it provide to save and restore the previous state of an object without revealing the details of its implementation.

3.5 Observer method

Observer Method is a Behavioral Design Pattern, it defines a one-to-many dependency between objects, so that when one object (the subject) changes state, all its dependents (observers) are notified and updated automatically.

3.6 State Method

State Method is a Behavioral Design Pattern, it allows an object to alter its behavior when its internal state changes.

3.7 Strategy Method

Strategy Method is a Behavioral Design Pattern, it defines a family of algorithms, encapsulates each one, and makes them interchangeable and it allows a client to choose an appropriate algorithm from a family of algorithms at runtime.

3.8 Template Method

Template Method is a Behavioral Design Pattern, it defines the skeleton of an algorithm in a method but lets subclasses alter some steps of that algorithm without changing its structure.

3.9 Visitor Method

Visitor Method is a Behavioral Design Pattern, it is used when you have a set of structured, hierarchical objects and you want to perform various operations on these objects without modifying their classes.

3.10 Interpreter Design Pattern

Interpreter pattern is used to define a grammatical representation for a language and provides an interpreter to deal with this grammar.

Knowing when to use design patterns in Python(or any programming language) is crucial for effective software design. Below are guidelines on when to use and when not to use design patterns:

When to Use Design Patterns in Python

- **Recurring Problems:** Use design patterns when you encounter recurring design problems that have well-established solutions. Design patterns provide tested and proven approaches to common software design challenges.
- **Flexibility and Reusability:** Use design patterns to promote code reusability, flexibility, and maintainability. They help in structuring code in a way that makes it easier to modify and extend as requirements evolve.
- **Design Principles:** Use design patterns to apply fundamental design principles such as separation of concerns, encapsulation, and dependency inversion. They help in achieving better modularity and reducing dependencies between components.
- **Communication:** Use design patterns to improve communication among team members. Design patterns provide a common vocabulary and understanding of how to solve particular problems, facilitating collaboration and code comprehension.
- **Performance:** In some cases, design patterns can improve performance by optimizing resource usage, reducing overhead, or improving code execution efficiency.

When not to Use Design Patterns in Python

- **Over-Engineering:** Avoid using design patterns unnecessarily, especially for small or simple problems. Over-engineering can lead to unnecessary complexity and overhead, making the codebase harder to understand and maintain.
- **Premature Optimization:** Avoid using design patterns solely for the sake of optimization before performance issues are identified. Premature optimization can lead to added complexity without significant benefits and can hinder future changes.
- **Unfamiliarity:** Avoid using design patterns if you or your team are unfamiliar with them or if their application does not align with the problem at hand. Using patterns incorrectly can lead to misuse and potential design flaws.

- **Project Constraints:** Consider project constraints such as time, budget, and team expertise. If applying a design pattern significantly increases development time or introduces unnecessary complexity, it may not be appropriate for the project.
- **Changing Requirements:** Be cautious when applying design patterns in highly dynamic environments where requirements frequently change. Overly rigid designs based on patterns may struggle to adapt to evolving requirements.

Pseudocode:

1. Define a `PaymentStrategy` interface with a method `pay`.
2. Implement various payment strategies (`CreditCardPayment`, `PayPalPayment`, `BitcoinPayment`) inheriting from `PaymentStrategy`.
3. Create a `PaymentProcessor` class that uses a `PaymentStrategy` to process payments.
4. Allow the `PaymentProcessor` to switch strategies at runtime.

Experiment 3 Title:**Experiment 3 Code:****Experiment 3 Output (Screenshot):****Bucket List Experiment Title:****Bucket List Experiment Code:****Bucket List Experiment Output (Screenshot):****Theory Questions (Handwritten):**

Q 1: Explain the Singleton design pattern in Python. How does it ensure that only one instance of a class is created? Provide a basic code example to demonstrate its implementation.

Q 2: Describe the Factory design pattern in Python. How does it differ from directly creating objects? Provide a simple example of a factory method that produces different types of objects based on input.

Conclusion:

The Strategy Pattern provides a flexible and scalable way to manage different payment methods in a payment processing system. By encapsulating each payment method within its own class, the system can easily switch between different algorithms at runtime without modifying the client code. This design pattern promotes the Open/Closed Principle, making the system open for extension but closed for modification.